

研究生项目指导书

AI-first LQCD Meson DA Analysis Agent

三个月高级训练与原型系统建设方案

项目导师版 / 学生执行版

2026 年 5 月 15 日

核心定位

本项目不是简单让学生“用 AI 写代码”，也不是单纯搭一个聊天机器人。目标是训练学生在 Codex 等 AI 工具帮助下，把真实的格点 QCD meson DA 数据分析流程尽可能交给 AI 自动执行、自动检查、自动复现和自动记录，只在关键决策、风险升级和人类主动介入时才引入人工判断，最终形成一个可复现、可审计、可扩展、AI-first、human-gated 的科研分析 agent 原型。

三个月结束时，学生应至少完成一个初级但完整的高级原型：以一个真实 golden example 为基准，能够让 AI 自动完成数据审计、配置生成、分析执行、结构化输出、质量检查、Supervisor 推荐、Skeptic 审查、case memory 更新，并在必须人工介入的少数关键节点触发 human approval gate；系统还应能通过 Snakemake 管理可复现 workflow，通过 LangGraph 管理多 agent 状态流转。

目录

1 项目总目标	4
1.1 科研工具目标	4
1.2 学生训练目标	4
2 当前仓库的真实 example	5
2.1 当前 example 的数据布局	5
2.2 当前 example 的真实分析顺序	5
2.3 学生应该先读哪些参考文档	6
2.4 这套 example 对学生意味着什么	6
2.5 Week 1: 真实 example 与统计物理基础	7
3 为什么采用三层架构	8
3.1 Layer 1: Analysis Core	8

3.2	Layer 2: Scientific Workflow Orchestration	8
3.3	Layer 3: Agent Orchestration	9
4	借鉴先进 multi-agent practice	9
4.1	Specialized Roles: 不要一个 agent 做所有事	9
4.2	Debate / Critic: 必须设计反方角色	10
4.3	Risk Gate: 不同风险等级使用不同审批规则	11
4.4	Memory: 不是简单保存聊天记录	11
5	三个月总体路线图	11
6	Month 1: 复现 golden example 与工程化基础	12
6.1	Month 1 目标	12
6.2	Month 1 学习内容	12
6.3	Month 1 任务	13
6.3.1	任务 1: 建立 repo skeleton	13
6.3.2	任务 2: 把 notebook 或散乱脚本变成 CLI	13
6.3.3	任务 3: 定义 config schema	13
6.3.4	任务 4: 定义 result schema	14
6.3.5	任务 5: 加入 pytest	14
6.4	Month 1 交付物	15
6.5	Month 1 验收标准	15
7	Month 2: Snakemake workflow 与 rule-based agents	15
7.1	Month 2 目标	15
7.2	Month 2 学习内容	15
7.3	Month 2 workflow 设计	16
7.4	Worker Agent wrapper	16
7.5	Fit Analyst	17
7.6	Skeptic Reviewer	17
7.7	Supervisor 与 decision log	18
7.8	Month 2 交付物	18
7.9	Month 2 验收标准	19
8	Month 3: LangGraph 多 agent 原型与高级展示	19
8.1	Month 3 目标	19
8.2	Month 3 学习内容	19
8.3	推荐 LangGraph state	19
8.4	推荐 graph	20
8.5	Human approval 不只是最后按钮	20
8.6	Agent evaluation harness	21

8.7	Month 3 交付物	21
8.8	Month 3 验收标准	21
9	Codex 使用规范	22
9.1	Codex 可以做什么	22
9.2	Codex 不应该直接决定什么	22
9.3	标准 Codex prompt 模板	23
10	推荐最终 repo 结构	23
11	每月汇报模板	25
12	导师检查清单	26
12.1	Month 1 检查	26
12.2	Month 2 检查	26
12.3	Month 3 检查	26
13	三个月后的高级扩展方向	26
14	参考资料与工具选择依据	27
15	最终要求	28

1 项目总目标

1.1 科研工具目标

本项目的长期目标是建立一个面向格点 QCD meson distribution amplitude 数据分析的 AI-first analysis agent 系统。该系统应满足以下要求：

- **可复现**：所有分析由 config、schema、workflow、日志和版本控制驱动，而不是依赖手动 notebook 操作。
- **可追责**：每一个重要判断都要留下 decision log，包括输入、候选方案、选择理由、风险等级和是否需要人工批准。
- **AI-first automation**：优先让 AI 自动完成重复性、结构化、可检查的科研工程工作，例如数据整理、workflow 执行、拟合扫描、图表生成、表格汇总和日志记录。
- **human-gated**：只有在关键决策、风险升级、异常诊断、结果 promotion、系统误差判断或人类主动要求介入时，才把任务交给人工确认。
- **逐步增强 AI 物理判断**：第一阶段以 rule-based Supervisor 和 structured checks 为主，后续逐步引入 LLM-based reasoning、multi-agent debate、case memory 和 reviewer critique。
- **真实科研可用**：最终应能让 AI 尽可能承担真实 meson DA 分析中的常规工作，而不是停留在玩具示例。

这里的核心理念是：先进的科研 AI agent 系统不是“工具越多越先进”，而是把科研流程拆解为可执行、可验证、可回放、可批注、可积累经验的结构化系统，并尽量把重复、机械、耗时的工作从人类手里移交给 AI。

1.2 学生训练目标

本项目同时是一个研究生训练项目。学生不需要一开始精通所有工具，但需要在三个月内逐步掌握现代 AI-first scientific workflow 的关键能力：

- 把 notebook-style 分析改造成 config-driven pipeline，并让 AI 自动处理能自动处理的大部分步骤；
- 用 Pydantic 或等价工具定义输入输出 schema；
- 用 pytest 验收 Codex 生成的代码；
- 用 Snakemake 管理依赖关系和增量重跑；
- 用 LangGraph 表达多 agent 状态流、人工中断和 targeted rerun；
- 用 Git 做小步提交，而不是一次性大改；
- 写清楚 Codex prompt，学会让 AI 生成代码、测试和文档，并尽量减少人类手工写实现；
- 理解 Worker、Fit Analyst、Skeptic Reviewer、Supervisor、Human Approval、Memory 的不同角色，以及哪些工作应该默认交给 AI，哪些必须留给人类；

- 初步理解 LQCD meson DA 分析中的 fit strategy、sanity check 和 physical interpretation。

三个月后，学生不一定能独立完成所有物理判断，但应能成为一个**科研 AI agent 搭建初级专家**：知道如何拆任务、如何让 AI 尽量自动化执行、如何测试、如何管理 workflow、如何设计 agent state、如何记录决策、如何在关键节点保留人工审查，以及如何向导师展示一个可靠的初级系统。

2 当前仓库的真实 example

为了避免学生一开始学到的是抽象口号而不是实际流程，本项目现在以仓库中的 `examples/l64c64a076_m140/` 作为贯穿全文的真实 example。这个 example 不是 toy data，而是一套已经裁剪成 DA-only 目标的实际分析链，包含当前分支要教给学生的最小闭环。

2.1 当前 example 的数据布局

当前 example 目录的核心内容是：

目录	作用
<code>examples/l64c64a076_m140/data/c2pt_csv/</code>	两点 correlator 的原始 CSV 输入
<code>examples/l64c64a076_m140/data/qda/</code>	只保留 <code>b_X</code> 、 <code>OT5</code> 、 <code>bT=0</code> 的 qDA HDF5 输入
<code>examples/l64c64a076_m140/analysis/0-effective-mass/</code>	effective mass 的最小练习，只做统计诊断和画图
<code>examples/l64c64a076_m140/analysis/1-c2pt-fit/</code>	两点拟合 notebook 和输出
<code>examples/l64c64a076_m140/analysis/2-bm/</code>	DA bare matrix element 提取
<code>examples/l64c64a076_m140/analysis/3-normalize/</code>	DA 归一化，当前 example 只保留 mode3 路径
<code>examples/l64c64a076_m140/analysis/4-FT/</code>	DA Fourier 变换，当前 example 只保留 mode3 路径
<code>examples/l64c64a076_m140/input_files/</code>	与 notebook 一致的纯文本输入模版

2.2 当前 example 的真实分析顺序

学生要记住的不是“有很多目录”，而是这条固定顺序：

阶段	结果层级	关键问题
0-effective-mass	effective mass 和图	plateau、噪声、bootstrap 误差是否合理
1-c2pt-fit	两点拟合	能量、overlap、 χ^2/dof 是否稳定
2-bm	DA bare matrix element	先把矩阵元从 ratio 中提出来
3-normalize	归一化后的 x-space/中间量	只保留 mode3 路径，检查归一化是否合理
4-FT	Fourier 结果	看最终分布的形状和物理趋势

这条链的关键约束现在是：

- 只保留 b_X；
- 只保留 OT5；
- 只保留 bT = 0；
- 删除 b_Y；
- 删除 OZ5；
- 下游 example 只保留 mode3 路径。

2.3 学生应该先读哪些参考文档

如果学生第一次接触这套流程，建议先把本章读完，再按当前 example 跑一遍；两点拟合和 DA ratio 的详细推导已经合并进 Month 1 指南，不需要再单独翻旧文件。

- docs/analysis_DA_agent/lqcd_agent_month1_detailed_guide_CN.pdf：包含两点拟合、bootstrap、最小 χ^2 、DA ratio-fit、输入方式和运行顺序的完整说明；
- examples/README.md：当前 example 的目录约定；
- templates/input_files/two_point/ 和 templates/input_files/da/：真实输入模版；
- templates/two_point/ 和 templates/da/：notebook 模版。

2.4 这套 example 对学生意味着什么

这套 example 的教学目标不是让学生“会点 notebook 按钮”，而是让学生能回答：

1. 这条分析链里每一步输入是什么；
2. 每一步输出是什么；
3. 每一步的物理意义是什么；
4. 哪些步骤是数值问题，哪些步骤是物理判断；

5. 哪些东西应该进 config, 哪些东西应该进 result;
6. 为什么最终结果不能只看一个数, 而要看整条 workflow 的 provenance。

2.5 Week 1: 真实 example 与统计物理基础

Week 1 是学生正式进入这套项目时的第一堂练习课。它的任务非常明确: 不要先做复杂拟合, 不要先学 agent, 而是先把 **effective mass** 这件最基础的事学明白。换句话说, Week 1 的目标是先建立 plateau、噪声和 bootstrap 的最基本直觉, 再去看后面的两点拟合。

在格点 QCD 的两点函数分析里, effective mass 是学生最先应该掌握的诊断量, 因为它能把原始 correlator 的指数衰减压成更容易观察的局部斜率图像。只要学生能看懂 effective mass 图, 后面的两点拟合、窗口选择和 DA ratio 才有意义。

从教学上说, Week 1 是一个最小闭环:

1. 读入一组两点 correlator;
2. 只计算 effective mass;
3. 只画图, 不做复杂拟合;
4. 从图上判断哪里开始接近单态区;
5. 把 plateau 和噪声结构带到后面的 1-c2pt-fit。

Week 1 的推荐顺序是:

1. 先打开 `examples/164c64a076_m140/analysis/0-effective-mass/`;
2. 先跑 notebook, 观察 effective mass 的 bootstrap 结果和图;
3. 再跑 input file, 确认同一组参数会得到同样的 effective mass 输出;
4. 重点看 plateau、噪声增长和误差条;
5. 记录你认为适合进入 1-c2pt-fit 的时间窗;
6. 再把这份观察带到后面的 Month 1 指南里。

学生在 Week 1 至少要回答下面几个问题:

- 为什么 effective mass 在小 t 和大 t 的行为不同;
- 哪些点最接近 plateau;
- 误差条什么时候开始快速变大;
- 为什么 effective mass 只能作为诊断, 而不是最终物理结果;
- 为什么不同动量的图不能机械地用同一个窗口。

如果学生连这一步都看不懂, 就不要急着进入两点拟合。先把 0-effective-mass 跑通, 是为了建立最基本的统计物理直觉, 而不是为了得到一个最终数值。

3 为什么采用三层架构

本项目采用三层架构，而不是把所有功能混在一个大 agent 中。

3.1 Layer 1: Analysis Core

Analysis Core 是最底层的科学计算代码。它不应该依赖 LLM，也不应该依赖复杂 agent。它只负责确定性的分析计算：

- 读取数据；
- 检查数据完整性；
- bootstrap / jackknife；
- effective mass；
- 2pt fit；
- ratio construction；
- ratio fit；
- bare matrix element extraction；
- plotting；
- 输出 JSON / CSV / PDF / log。

这一层必须尽量简单、清楚、可测试。任何 agent 都不应该直接把未经测试的复杂逻辑塞进这一层。

3.2 Layer 2: Scientific Workflow Orchestration

这一层负责管理分析步骤之间的依赖关系。这里建议使用 **Snakemake** 作为主 workflow engine。

选择 Snakemake 的理由是：

- 它是成熟的科学 workflow 系统，目标正是支持 reproducible and scalable data analyses；
- 它非常适合文件驱动的科研流程，例如 raw data $\rightarrow$ fit result $\rightarrow$ plots $\rightarrow$ summary；
- 它用 Python 风格规则描述 workflow，适合 LQCD 学生逐步学习；
- 它适合 HPC 环境和批量 ensemble / momentum / z / fit-window 扫描；
- 它可以避免“改了一个 config 以后不知道哪些步骤要重跑”的问题。

本项目不选择 Dagster 或 Prefect 作为第一主线。Dagster 的 software-defined asset 理念很先进，Prefect 的 Python-native retry 和 observability 也很好，但它们更偏数据工程平台或服务化任务编排。对于当前 LQCD 文件型分析流程，Snakemake 更贴近科研工作流和 HPC 实践。Dagster / Prefect 可作为第二阶段生产化或 dashboard 化的候选方案。

3.3 Layer 3: Agent Orchestration

这一层负责多 agent 状态流、人工审批、异常回退、targeted rerun 和 memory update。这里建议使用 **LangGraph** 作为主 agent orchestration framework。

选择 LangGraph 的理由是：

- 它适合构建 stateful multi-agent workflow；
- 它支持 durable execution，即执行状态可以持久化；
- 它适合 human intervention interrupt / resume；
- 它可以把 agent 流程表示为清楚的 graph，而不是不可追踪的长聊天；
- 它适合实现 Data Auditor → Worker → Fit Analyst → Skeptic Reviewer → Supervisor → Human Approval → Memory 的闭环。

重要的是：LangGraph 不替代 Snakemake。二者分工不同：

工具	负责什么	不负责什么
Snakemake	科学计算 workflow、文件依赖、增量重跑、HPC 批量执行	不负责 agent reasoning 和 human approval 对话
LangGraph	多 agent 状态流、判断节点、人工中断、targeted rerun、memory update	不负责底层文件依赖和大规模批处理

因此本项目的推荐结构是：**Snakemake 管数据分析 DAG，LangGraph 管 agent 决策 DAG。**

4 借鉴先进 multi-agent practice

本项目借鉴现代 multi-agent 系统中的几个关键思想，尤其是类似 TradingAgents 这类系统中的 specialized roles、debate、risk management 和 final decision gate。

4.1 Specialized Roles: 不要一个 agent 做所有事

一个大而全的 agent 很容易出错，因为它同时要读数据、写代码、判断 fit、解释物理、更新 memory。更先进的设计是把任务拆成多个有明确边界的角色：

角色	主要责任	禁止事项
Data Auditor	检查文件、metadata、缺失数据、shape、ensemble/momentum/z 覆盖	不做 fit 选择

Worker Agent	生成 config、调用 CLI、写缺失脚本、修 bug、加测试、产出结果	不决定 central result
Fit Analyst	比较 1-state/2-state/3-state、fit window、prior、plateau、 χ^2/dof	不绕过 Reviewer
Skeptic Reviewer	专门找问题：不稳定、过拟合、异常 dependence、normalization issue	不直接批准最终结果
Supervisor	综合 Fit Analyst 和 Skeptic 意见，给出推荐和风险等级	高风险结果不能自动通过
Human Approver	导师或高级研究者最终批准高风险 physics decision	不需要检查所有低风险机械步骤
Memory Manager	记录 accepted/rejected cases、失败原因、经验规则	不写入未经批准的 central result

4.2 Debate / Critic: 必须设计反方角色

先进 multi-agent 系统通常不是让一个 agent 直接给结论，而是让不同角色从不同角度分析。对于 LQCD 分析，Skeptic Reviewer 很重要。

例如 Fit Analyst 可能说：

2-state fit 在 $t_{\min} = 4$ 到 6 之间稳定， χ^2/dof 可接受，因此推荐 2-state central fit。

Skeptic Reviewer 必须检查：

- 1-state 和 2-state 是否在误差内一致？
- 结果是否强烈依赖 $t_{\min}$ ？
- posterior 是否主要由 prior 决定？
- ratio fit 是否和 2pt fit 的能隙假设一致？
- local matrix element 是否合理？
- P_z dependence 是否异常？
- z -dependence 是否平滑？
- real/imag symmetry 是否符合预期？

这不是“让 AI 吵架”，而是把科研审查制度化。

4.3 Risk Gate: 不同风险等级使用不同审批规则

所有 decision 都应分风险等级。

风险等级	例子	处理方式
Low	文件齐全、测试通过、fit 稳定、sanity check 正常	可自动通过并记录
Medium	fit 稳定性一般、model dependence 可见、统计误差偏大	需要 Supervisor 明确理由，可请求人工复核
High	central fit 选择、丢弃数据、系统误差、异常物理解释、结果 promotion	必须 human approval

4.4 Memory: 不是简单保存聊天记录

Case memory 应保存结构化经验，而不是保存一堆对话。

推荐至少包含：

- case_id;
- ensemble / momentum / z / operator / channel;
- 使用的 config hash;
- 候选 fit 列表;
- accepted fit;
- rejected fit 及原因;
- risk level;
- human approval 状态;
- downstream validation 结果;
- lessons learned。

初期可用 YAML / JSON。三个月后建议迁移到 SQLite 或 DuckDB，以支持按 ensemble、momentum、 z 、失败类型检索。

5 三个月总体路线图

三个月计划分为三个阶段：

- **第 1 个月：复现与工程化。**目标是让学生理解现有真实分析流程，并把 golden example 变成可测试、可复现的 config-driven CLI。
- **第 2 个月：workflow 与 agent 化。**目标是用 Snakemake 管理分析 DAG，用 rule-based agents 形成可审计的 Worker / Fit Analyst / Skeptic / Supervisor 流程。

- **第 3 个月: LangGraph 多 agent 原型与高级展示。**目标是将第二个月形成的稳定 CLI 和 rule-based 逻辑接入 LangGraph, 实现 stateful multi-agent prototype、human approval、targeted rerun 和 case memory。

阶段	核心任务	最终产物
Month 1	Golden example 复现、CLI、schema、pytest、Codex 训练	可运行 analysis core
Month 2	Snakemake workflow、Worker wrapper、rule-based Supervisor、decision log	可复现 workflow + 初级 agent 判断
Month 3	LangGraph orchestration、Skeptic debate、human approval、memory、demo report	高级多 agent 原型

6 Month 1: 复现 golden example 与工程化基础

6.1 Month 1 目标

第一个月的目标不是从零发明新 physics, 而是把一个已经准备好的实际分析流程变成学生能复现、能测试、能解释、能扩展的工程化系统。

建议选择一个真实的 golden example, 例如:

```
examples/l64c64a076_m140/
```

它应包含一套相对完整、已知能跑通的 meson DA 分析输入输出。这个 example 同时承担三个作用:

- **golden example:** 作为学生第一套可复现案例;
- **regression example:** 以后代码修改后必须仍能复现;
- **agent training example:** 用于训练 Supervisor 和 Skeptic 判断什么是正常、什么是异常。

6.2 Month 1 学习内容

学生需要学习:

- Python package 基本结构;
- 命令行 CLI;
- YAML / JSON config;
- Pydantic schema;

- pytest;
- Git 小步提交;
- Codex prompt 写法;
- meson DA 分析流程的输入输出结构;
- 2pt fit、ratio、bare matrix element 的基本含义。

6.3 Month 1 任务

6.3.1 任务 1: 建立 repo skeleton

推荐目录:

```
lqcd-meson-da-agent/  
  README.md  
  pyproject.toml  
  configs/  
    l64c64a076_m140.yaml  
  examples/  
    l64c64a076_m140/  
  src/  
    lqcd_agent/  
      __init__.py  
      schema/  
      analysis/  
      io/  
      plots/  
  tests/  
  docs/  
  outputs/  
  .gitkeep
```

学生应使用 Codex 生成初版, 但必须检查每个文件是否合理。

6.3.2 任务 2: 把 notebook 或散乱脚本变成 CLI

目标命令:

```
python -m lqcd_agent.analysis.run \  
  --config configs/l64c64a076_m140.yaml
```

CLI 不一定一开始完成所有真实分析, 但必须形成稳定接口。

6.3.3 任务 3: 定义 config schema

示例:

```
class EnsembleConfig(BaseModel):
    name: str
    lattice_spacing: float | None = None
    pion_mass: float | None = None
    base_dir: str

class AnalysisConfig(BaseModel):
    case_id: str
    ensemble: EnsembleConfig
    momenta: list[str]
    z_values: list[int]
    channels: list[str]
    output_dir: str
    seed: int = 1234
    allow_overwrite: bool = False
```

原则：所有会影响结果的参数都必须进入 config，不能散落在代码中。

6.3.4 任务 4：定义 result schema

至少包括：

- input config path;
- config hash;
- git commit hash;
- output files;
- candidate fits;
- summary statistics;
- warnings / errors;
- run status。

6.3.5 任务 5：加入 pytest

Month 1 至少需要：

- import test;
- config validation test;
- CLI smoke test;
- golden example output existence test;
- no-overwrite safety test。

6.4 Month 1 交付物

- 可安装或可导入的 Python repo;
- 一个 golden example config;
- 一个可运行 CLI;
- 初版 Pydantic schema;
- 初版 pytest;
- 输出 `result.json`、`summary.csv`、`plots`、`run.log`;
- 一份 Month 1 report。

6.5 Month 1 验收标准

学生必须能做到：

- 从 clean repo 运行一条命令复现 golden example;
- 解释 config 中每个字段;
- 解释 `result.json` 中每个字段;
- 运行 `pytest` 并说明每个测试保护什么;
- 展示至少 8-12 个小 commit;
- 说明 Codex 生成了什么、自己检查了什么、改了什么。

7 Month 2: Snakemake workflow 与 rule-based agents

7.1 Month 2 目标

第二个月的目标是把 Month 1 的 CLI 分析核心升级为可复现 workflow，并加入第一版 rule-based agents。

这里正式引入 Snakemake。

7.2 Month 2 学习内容

学生需要学习：

- Snakemake rule、input、output、params、shell / script;
- dependency-aware rerun;
- workflow config;
- Worker wrapper;
- rule-based Fit Analyst;

- Skeptic Reviewer;
- Supervisor decision log;
- case memory。

7.3 Month 2 workflow 设计

推荐 Snakemake DAG:

```
audit_data
-> generate_config
-> run_2pt_fit
-> run_ratio_analysis
-> collect_bare_matrix_elements
-> make_plots
-> collect_summary
-> supervisor_decision
-> report
```

示例文件结构:

```
workflow/
  Snakefile
  config.yaml
  rules/
    audit.smk
    fit.smk
    ratio.smk
    plots.smk
    supervisor.smk
```

7.4 Worker Agent wrapper

Worker 的输入应是结构化 task:

```
{
  "task_id": "run_l64c64a076_m140_001",
  "task_type": "run_analysis_workflow",
  "config_path": "configs/l64c64a076_m140.yaml",
  "output_dir": "outputs/l64c64a076_m140",
  "allow_overwrite": false
}
```

Worker 输出:

```
{
  "task_id": "run_l64c64a076_m140_001",
  "status": "success",
```



```
"outputs": {
  "result_json": "outputs/.../result.json",
  "summary_csv": "outputs/.../summary.csv",
  "plots_dir": "outputs/.../plots",
  "run_log": "outputs/.../run.log"
},
"warnings": [],
"errors": []
}
```

Worker 必须遵守：

- 不直接修改 raw data；
- 默认不覆盖已有输出；
- 所有命令写入 log；
- 所有失败写入 structured error；
- 重大代码修改必须附带 tests；
- 不做 central physical result 决策。

7.5 Fit Analyst

Fit Analyst 负责对候选 fit 做结构化分析：

- 读取 summary.csv；
- 比较不同 fit window；
- 比较 1-state / 2-state / 3-state；
- 检查 χ^2/dof ；
- 检查 fit stability；
- 生成少量 high-value fit recommendation；
- 避免 brute-force scan。

7.6 Skeptic Reviewer

Skeptic Reviewer 的任务是找问题，不是给好消息。

它应检查：

- fit 是否对 $t_{\min}$ 过度敏感；
- 2pt fit 是否可能污染 ratio fit；
- local matrix element 是否异常；

- P_z dependence 是否异常;
- z -dependence 是否不平滑;
- real/imaginary symmetry 是否异常;
- normalization 是否可能出错;
- 是否需要 targeted rerun。

7.7 Supervisor 与 decision log

Supervisor 综合 Fit Analyst 和 Skeptic Reviewer 的结果, 输出:

```
{
  "case_id": "l64c64a076_m140",
  "decision_type": "fit_recommendation",
  "candidate_fits": [...],
  "recommended_fit": "...",
  "skeptic_flags": [...],
  "risk_level": "medium",
  "human_approval_required": true,
  "reasons": [
    "2-state fit is more stable than 1-state fit",
    "local matrix element passes sanity check",
    "Pz dependence needs human review"
  ],
  "targeted_rerun": {
    "needed": false,
    "reason": null
  }
}
```

7.8 Month 2 交付物

- Snakemake workflow;
- Worker wrapper;
- Fit Analyst rule module;
- Skeptic Reviewer rule module;
- Supervisor decision module;
- decision_log.json;
- 初版 accepted_cases.yaml / rejected_cases.yaml;
- tests for Worker / Supervisor / Skeptic;
- Month 2 report。

7.9 Month 2 验收标准

学生必须能做到：

- 用 `snakemake --cores 1` 复现 golden example；
- 修改一个输入或 config 后，只重跑必要下游步骤；
- Worker 能安全执行 workflow；
- Supervisor 能输出 decision log；
- Skeptic 能指出至少三类潜在问题；
- 至少有三种测试案例：good case、ambiguous case、failed case；
- 学生能解释 Snakemake 和 agent 的区别。

8 Month 3: LangGraph 多 agent 原型与高级展示

8.1 Month 3 目标

第三个月正式把 rule-based agent pipeline 接入 LangGraph, 实现 stateful multi-agent prototype。

注意：LangGraph 不是用来替代已有 CLI 或 Snakemake，而是用来组织 agent 决策流。

8.2 Month 3 学习内容

学生需要学习：

- LangGraph StateGraph；
- typed state；
- node / edge / conditional edge；
- checkpoint / durable execution；
- human interrupt / resume；
- targeted rerun loop；
- memory update；
- agent evaluation harness；
- final demo report。

8.3 推荐 LangGraph state

```
class AnalysisAgentState(TypedDict):
    case_id: str
    config_path: str
    output_dir: str
    audit_report: dict | None
    worker_result: dict | None
    fit_analysis: dict | None
    skeptic_review: dict | None
    supervisor_decision: dict | None
    human_feedback: dict | None
    memory_update: dict | None
    errors: list[dict]
    status: str
```

8.4 推荐 graph

```
START
-> data_auditor
-> worker_run_snakemake
-> fit_analyst
-> skeptic_reviewer
-> supervisor
-> risk_gate
    -> if low risk: memory_update -> END
    -> if medium/high risk: human_approval
        -> if approved: memory_update -> END
        -> if request_rerun: targeted_rerun -> worker_run_snakemake
        -> if rejected: memory_update -> END
```

8.5 Human approval 不只是最后按钮

Human approval 应该出现在关键风险节点，例如：

- central fit selection;
- fit window 变更;
- 数据丢弃;
- systematic uncertainty 估计;
- accepted result promotion;
- paper-level conclusion。

学生应理解：human-gated 的高级实践不是“最后 approve 一下”，而是让人类在最需要判断的节点介入。

8.6 Agent evaluation harness

第三个月应加入一个最小 evaluation harness，用来测试 agent 决策是否稳定。
至少包含：

- good synthetic case;
- noisy case;
- unstable fit case;
- normalization problem case;
- missing data case;
- suspicious P_z dependence case。

每个 case 都应有 expected behavior，例如：

- good case: low risk, 自动通过;
- unstable fit case: medium/high risk, 请求人工批准;
- missing data case: Worker 不运行 fit, Data Auditor 报错;
- normalization problem: Skeptic flag normalization issue, Supervisor 建议 targeted rerun。

8.7 Month 3 交付物

- LangGraph prototype;
- typed agent state;
- graph nodes for Data Auditor / Worker / Fit Analyst / Skeptic / Supervisor / Human Approval / Memory;
- checkpoint 或可恢复执行机制;
- human approval demo;
- targeted rerun demo;
- case memory SQLite / DuckDB / YAML 版本;
- evaluation cases;
- final demo report;
- final presentation slides 或 markdown report。

8.8 Month 3 验收标准

学生必须能展示：

- 一个完整 LangGraph agent run;

- graph 中每个 node 的输入输出；
- 一次 high-risk case 的 human approval；
- 一次 Skeptic 触发 targeted rerun；
- decision log 和 memory update；
- 如何复现实验；
- 哪些地方仍然需要导师物理判断。

9 Codex 使用规范

9.1 Codex 可以做什么

Codex 适合：

- repo skeleton；
- CLI；
- Pydantic schema；
- pytest；
- Snakemake rule 初稿；
- LangGraph node skeleton；
- fake data / evaluation case generator；
- README；
- 小范围重构；
- traceback-based bug fix。

9.2 Codex 不应该直接决定什么

Codex 不应直接决定：

- central fit；
- 是否丢弃数据；
- systematic uncertainty；
- accepted result promotion；
- paper-level physical conclusion；
- 未经测试的大范围重构。

9.3 标准 Codex prompt 模板

We are building an AI-first LQCD meson DA analysis-agent system.
Current milestone: [Month 1 / Month 2 / Month 3].

Task:

[one small concrete task]

Files to edit:

[list files]

Requirements:

- [requirement 1]
- [requirement 2]
- [requirement 3]

Inputs:

[describe inputs]

Outputs:

[describe outputs]

Tests:

Please add or update pytest tests for:

- [test 1]
- [test 2]
- [test 3]

Constraints:

- Do not modify unrelated files.
- Do not change public interfaces unless necessary.
- Do not overwrite existing accepted outputs.
- Keep code simple and readable.
- If you introduce a new dependency, explain why.

After implementation, explain:

- what changed;
- how to run it;
- what tests were added;
- what still needs human review.

10 推荐最终 repo 结构

lqcd-meson-da-agent/

```
README.md
pyproject.toml
configs/
  164c64a076_m140.yaml
examples/
  164c64a076_m140/
src/
  lqcd_agent/
    schema/
    analysis/
    io/
    plots/
    worker/
    agents/
      data_auditor.py
      fit_analyst.py
      skeptic.py
      supervisor.py
      memory.py
    graph/
      state.py
      build_graph.py
      run_graph.py
  workflow/
    Snakefile
    config.yaml
    rules/
  tests/
    test_schema.py
    test_cli.py
    test_worker.py
    test_snakemake_smoke.py
    test_supervisor.py
    test_skeptic.py
    test_graph.py
  memory/
    accepted_cases.yaml
    rejected_cases.yaml
    cases.sqlite
  docs/
    student_guide.md
    architecture.md
    decision_policy.md
    codex_prompts.md
  outputs/
```



```
.gitkeep
```

11 每月汇报模板

Month X Report

1. This month's goal

[short paragraph]

2. Implemented features

- feature 1
- feature 2
- feature 3

3. Reproduction commands

[commands]

4. Main outputs

- output 1
- output 2
- output 3

5. Tests

pytest result: pass/fail

Important tests added:

- test 1
- test 2

6. Codex usage

- prompts used
- generated code reviewed
- bugs fixed

7. Scientific checks

- fit stability
- sanity checks
- skeptic flags

8. Failures and lessons

- issue 1
- lesson 1

9. Questions for advisor

- question 1

```
- question 2

10. Plan for next month
- task 1
- task 2
```

12 导师检查清单

12.1 Month 1 检查

- 是否能复现 golden example?
- config 是否完整?
- result schema 是否足够结构化?
- pytest 是否真正保护关键行为?
- 学生是否理解 basic analysis outputs?

12.2 Month 2 检查

- Snakemake 是否正确管理依赖?
- Worker 是否安全?
- Supervisor 是否记录理由?
- Skeptic 是否能指出真实风险?
- decision log 是否可读、可追责?

12.3 Month 3 检查

- LangGraph state 是否清楚?
- agent node 是否有明确输入输出?
- human approval 是否在正确节点?
- targeted rerun 是否真的闭环?
- memory 是否保存结构化经验?
- 学生是否能解释系统的边界和风险?

13 三个月后的高级扩展方向

三个月后，如果原型成功，可以继续扩展：

- 从 rule-based Supervisor 升级到 LLM-assisted Supervisor;

- 加入 multi-agent debate: Fit Analyst vs Skeptic;
- 引入 MLflow 或 OpenTelemetry 风格 tracing 记录 cost、latency、tool calls;
- 用 SQLite / DuckDB 做正式 case memory;
- 加入 RAG, 让 agent 读取项目 notes、fit policy、previous decisions;
- 支持多个 ensemble 的迁移;
- 支持 batch targeted rerun;
- 建立 dashboard 展示 analysis DAG、agent graph、decision log、plots;
- 加入真实 meson DA downstream validation;
- 最终形成 paper-quality analysis assistant。

14 参考资料与工具选择依据

本方案的工具选择基于以下原则:

- Snakemake 适合 reproducible and scalable scientific data analysis, 尤其适合文件依赖清楚的科研 workflow。
- LangGraph 适合 stateful agent workflow、durable execution 和 human intervention interrupt / resume。
- TradingAgents 类系统体现了 specialized roles、debate、risk management 和 final decision gate 的 multi-agent 设计思想。
- Dagster 的 software-defined assets 和 Prefect 的 Python-native retry / observability 很有价值, 但本项目第一主线更适合 Snakemake + LangGraph 的组合。
- Nextflow 在容器化、跨平台、标准化大 pipeline 中很强, 但对当前 Python-heavy、LQCD exploratory analysis 来说, Snakemake 学习成本和适配性更合适。

Online references

- TradingAgents project: <https://tradingagents-ai.github.io/>
- TradingAgents paper: <https://arxiv.org/abs/2412.20138>
- Snakemake documentation: <https://snakemake.readthedocs.io/>
- LangGraph durable execution documentation: <https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Nextflow documentation: <https://www.nextflow.io/docs/latest/>
- Prefect documentation: <https://www.prefect.io/how-it-works>
- Dagster software-defined assets: <https://docs.dagster.io/>

15 最终要求

三个月结束时，学生需要提交：

1. GitHub repo;
2. README;
3. golden example reproduction command;
4. Snakemake workflow;
5. LangGraph prototype;
6. pytest test suite;
7. decision logs;
8. memory examples;
9. final demo report;
10. 15-20 分钟 presentation。

最终展示时，学生应能清楚回答：

- 这个系统的输入是什么？
- 如何复现结果？
- 哪些步骤由 Worker 完成？
- 哪些判断由 Supervisor 完成？
- Skeptic Reviewer 找到了哪些风险？
- 哪些地方需要 human approval？
- decision log 如何记录理由？
- case memory 如何帮助下一次分析？
- 当前系统还不能做什么？
- 下一步如何扩展到更多 ensemble 或真实 production analysis？

给学生的最后提醒

本项目的目标不是让 AI 只做辅助性的零碎工作，而是让 AI 尽可能承担科研工程里的重复劳动、流程执行、初步检查和结果整理；人类只在关键决策、风险升级、异常解释和主动审核节点介入，同时用 schema、tests、workflow、logs、reviewer、approval 和 memory 保证科研可靠性。

你三个月后最重要的能力不是“会调用某个工具”，而是知道如何把一个真实科研分析问题拆成一个尽量自动化、可复现、可检查、可审计、只在必要时才需要人类介入的系统。